

6. Requirements

This section defines the **functional and non-functional requirements** for the Payment Gateway system. These requirements guide the overall architecture and design decisions.

6.1 Functional Requirements

These describe what the system **must do** from a user and business perspective.

1. Merchant Registration & Webhooks

- Merchants must be able to:
 - Register on the platform
 - Configure webhook endpoints (URLs to receive event notifications)
- Webhooks are used to notify merchants about:
 - Payment success/failure
 - Refunds
 - Payout updates

2. Payment Processing

- Customers must be able to:
 - Pay for goods/services using supported payment methods (primarily cards)
- System must:
 - Create and process a payment request (Payment Intent)
 - Authorize, capture, and settle payments securely

3. Merchant Notifications (Webhooks)

- Merchant must receive real-time updates about payment status:
 - Payment succeeded
 - Payment failed
 - Chargebacks / disputes (if applicable)
- Delivery guarantee:

- **At least once delivery**
- Retry mechanism for up to **30 days**

4. Payouts to Merchants

- Merchants must receive money in their **bank accounts**
- System should:
 - Aggregate successful payments
 - Deduct fees
 - Initiate payout transfers via acquiring bank
- Payouts can be:
 - Scheduled (daily/weekly)
 - On-demand (instant payout)

5. Idempotency / No Double Charge

- System must ensure:
 - A customer is never charged twice for the same transaction
- Achieved using:
 - Idempotency keys for payment requests
 - Deduplication at API and ledger level

6. Ledger (Audit Requirement)

- Every financial transaction must be stored in an **immutable ledger**
- Ledger must:
 - Record all debits and credits
 - Maintain full audit history
 - Support legal compliance and traceability
- Ledger is the **system of record**

6.2 Non-Functional Requirements

These define the **quality attributes** of the system.

1. High Traffic Scale

The system must handle large-scale traffic:

- **Payment Requests**
 - 500M/day $\approx$ ~5,787 RPS
 - Peak: ~10,000 RPS
- **Ledger Events**
 - 5B/day $\approx$ ~57,870 RPS
 - Peak: ~100,000 RPS
- **Merchants**
 - ~2 million active merchants

2. Scalability & Traffic Spikes

- Traffic is not uniform
- Must handle:
 - Seasonal spikes (Black Friday, sales events)
 - Merchant-specific bursts (high-traffic merchants)
- System should be **horizontally scalable**

3. Availability vs Consistency (CAP Tradeoff)

Payment Processing:

- Requires **Strong Consistency (Linearizability)**
- Must prevent:
 - Double charging
 - Race conditions
 - Partial payment states

Supporting Systems (less critical paths):

- Eventual consistency is acceptable for:
 - Merchant metadata
 - Reporting systems
 - Some ledger read replicas

4. Data Integrity & Correctness

- Ledger must always satisfy:
 - **Debit = Credit**
 - Total system balance must always be zero-sum
- Any inconsistency is unacceptable in financial systems

5. High Availability

- System should be:
 - Highly available (near 99.99%+)
 - Fault tolerant
- Payment system downtime directly impacts revenue, so:
 - Failures must degrade gracefully
 - Retry mechanisms must exist

6. Webhook Delivery Guarantees

- Webhooks must provide:
 - **At least once delivery**
 - Retry mechanism for failures
- Retry duration:
 - Up to **30 days**
- Ordering is not guaranteed, so events must be idempotent

7. No Data Loss (Ledger Requirement)

- Ledger is critical:
 - Must never lose entries
 - Must be append-only
- Even in failure scenarios:
 - Data must be recoverable
 - Consistency must be restored via reconciliation

6.3 Key Design Implications

These requirements directly influence system design:

- Need **distributed ledger system**

- Strong consistency for payments → likely use **synchronous DB / transactional system**
- Async processing for:
 - Webhooks
 - Payouts
 - Reporting
- Heavy use of:
 - Idempotency keys
 - Event queues
 - Retry mechanisms
- Partitioning needed to handle **100k+ RPS systems**

6.4 Summary

- System supports merchants, payments, payouts, and webhooks
- Must prevent double charges and ensure correctness
- Ledger is the legal source of truth
- Extremely high throughput system (10k+ RPS payments, 100k+ RPS ledger events)
- Strong consistency for money movement, eventual consistency elsewhere
- Must be highly available, fault-tolerant, and audit-compliant